Fine-Tuning LLMs

AutoML LoRA/QLoRA, Optuna, FAQ model tune

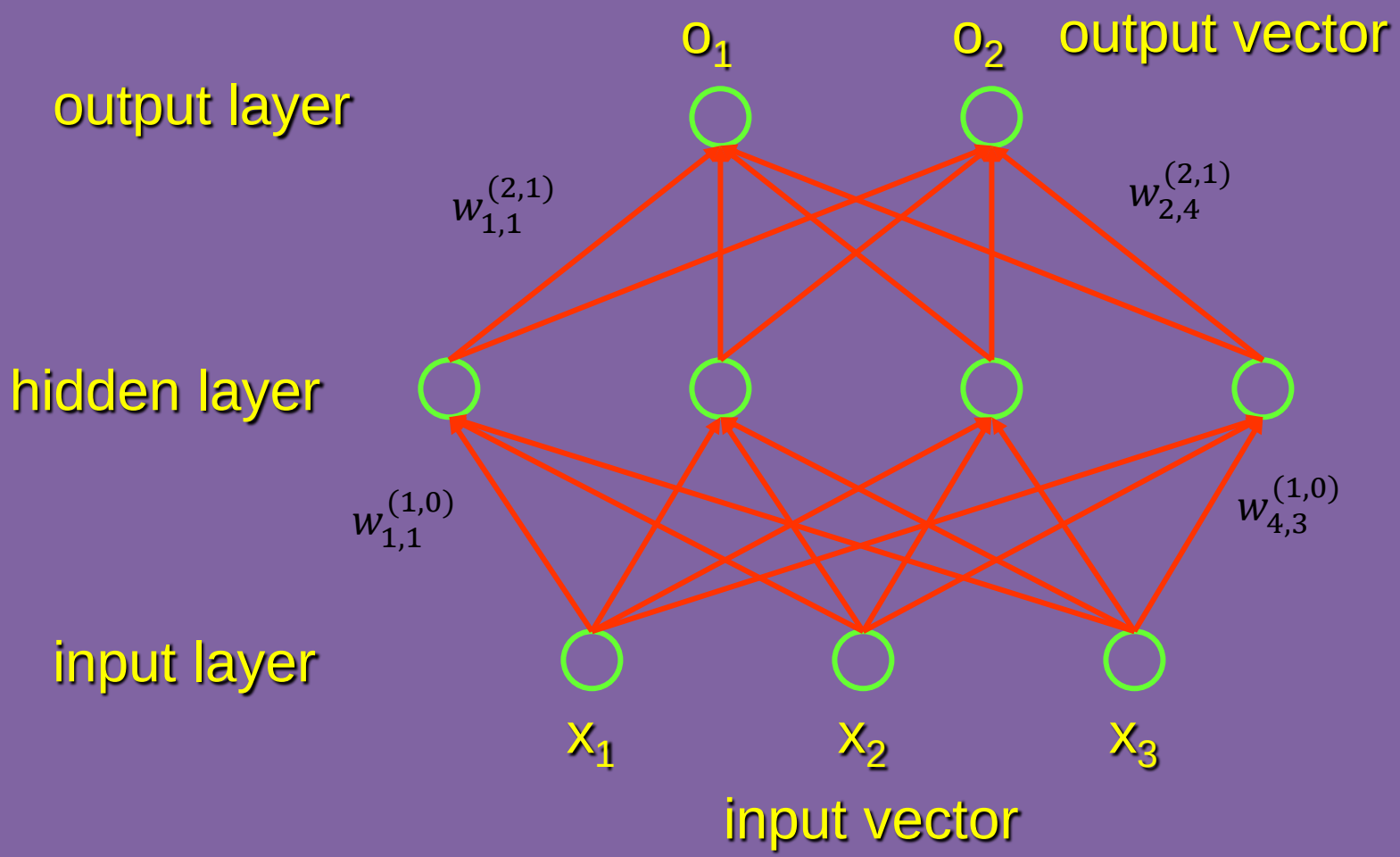
Part - 1

Prof. Surya Prakash

IIT Indore

Backpropagation Learning

- **Example:** Network function f: $\mathbf{R}^3 \rightarrow \mathbf{R}^2$



Backpropagation Learning

- For input pattern p, the i-th input layer node holds $x_{p,i}$.
- Net input to j-th node in hidden layer:

$$net_j^{(1)} = \sum_{i=0}^{n} w_{j,i}^{(1,0)} x_{p,i}$$

Output of j-th node in hidden layer:

$$x_{p,j}^{(1)} = S\left(\sum_{i=0}^{n} w_{j,i}^{(1,0)} x_{p,i}\right)$$

Net input to k-th node in output layer:

$$net_k^{(2)} = \sum_{j} w_{k,j}^{(2,1)} x_{p,j}^{(1)}$$

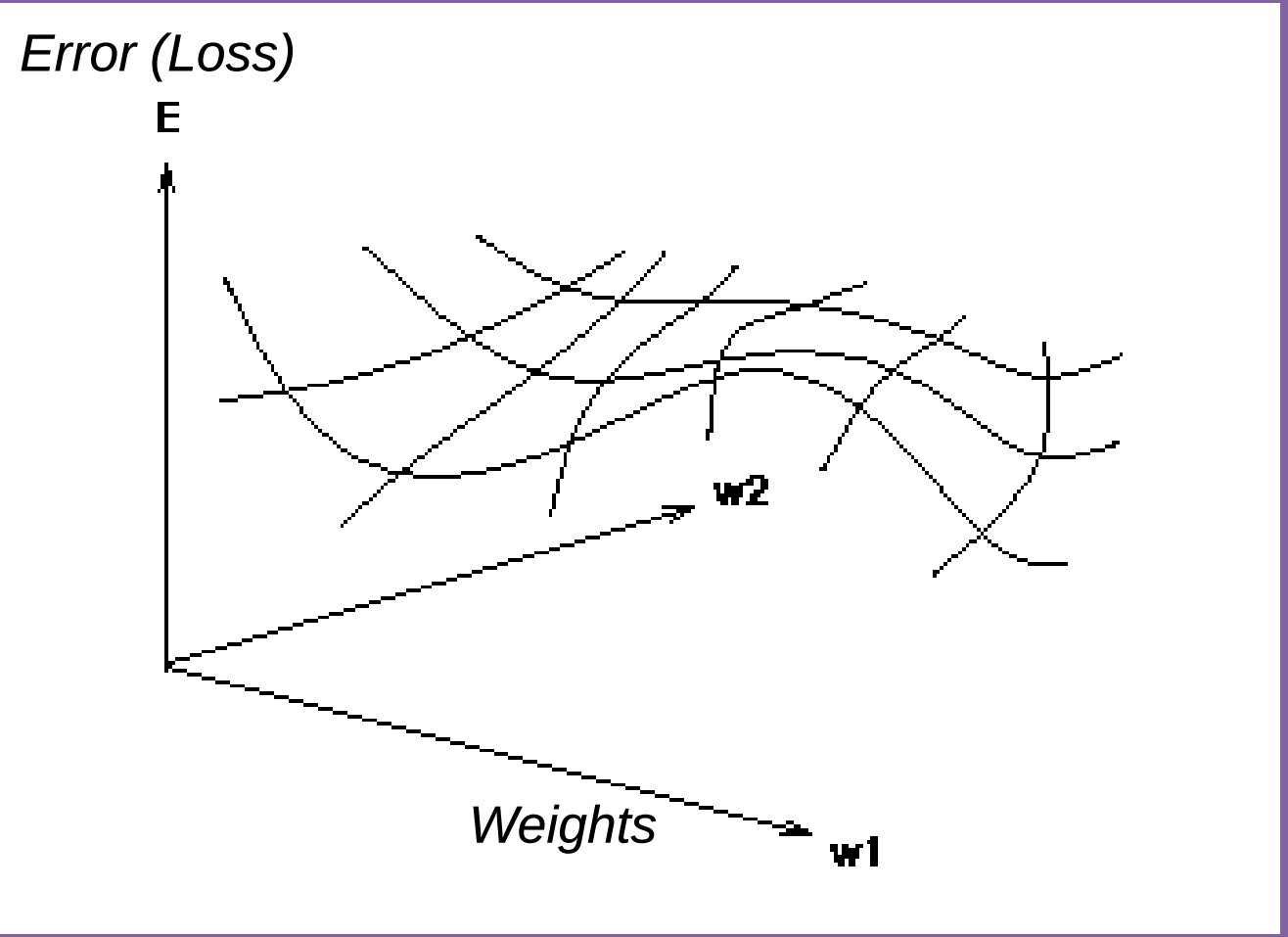
Output of k-th node in output layer:

$$o_{p,k} = S\left(\sum_{j} w_{k,j}^{(2,1)} x_{p,j}^{(1)}\right)$$

Network error (loss) for p:

$$L_p = \sum_{k=1}^{K} (l_{p,k})^2 = \sum_{k=1}^{K} (d_{p,k} - o_{p,k})^2$$

Error (Loss) Surface



$$L_p = f(w_{j,i},\ w_{k,j})$$

Error as function of weights in multidimensional space

Backpropagation Learning

- As E is a function of the network weights, we can use gradient descent to find those weights that result in minimal error.

- For individual weights in the hidden and output layers, we should move against the error gradient (*omitting index p to simplify the expression*):

$$\Delta w_{k,j}^{(2,1)} \propto \frac{-\partial L}{\partial w_{k,j}^{(2,1)}}$$

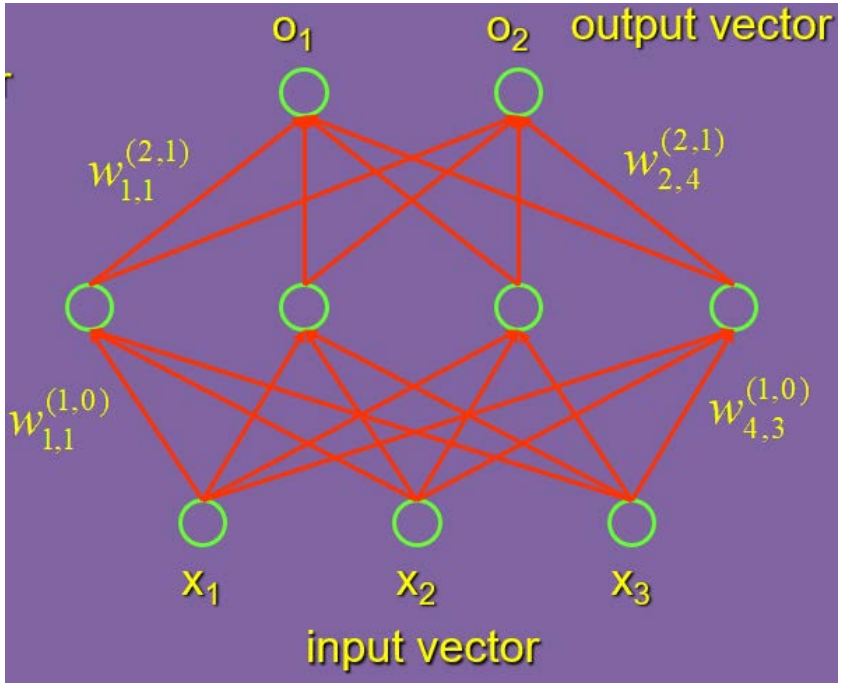
Output layer:
Derivative easy to calculate

$$\Delta w_{j,i}^{(1,0)} \propto \frac{-\partial L}{\partial w_{j,i}^{(1,0)}}$$

Hidden layer:
Derivative difficult to calculate

Backpropagation Learning

- Final weight update form



$$w_{k,j}^{(2,1)} \leftarrow w_{k,j}^{(2,1)} + \Delta w_{k,j}^{(2,1)}$$

where: $\Delta w_{k,j}^{(2,1)} = -\eta \frac{\partial L}{\partial w_{k,j}^{(2,1)}}$

$$w_{j,i}^{(1,0)} \leftarrow w_{j,i}^{(1,0)} + \Delta w_{j,i}^{(1,0)}$$

where $\Delta w_{j,i}^{(1,0)} = -\eta \frac{\partial L}{\partial w_{j,i}^{(1,0)}}$

1. Fine-Tuning LLMs

- Large language models (LLMs) like GPT, LLaMA, or Falcon are **pretrained** on huge corpora.

- Pretrained models are general-purpose

- Fine-tuning adapts them to **specific tasks**:
  - Sentiment analysis
  - Question answering (FAQ bots)
  - Summarization
  - Domain-specific knowledge (medical, legal)

Approaches to Fine-Tuning LLM

- Full Fine-Tuning
  - Update all model parameters
- LoRA (Low-Rank Adaptation)
  - Add small trainable matrices to existing weights
- QLoRA (Quantized LoRA)
  - LoRA + quantization (4-bit/8-bit)
- Prompt Tuning
  - **Add learnable soft prompt vectors at the input only** and keep the model fixed
- Prefix Tuning
  - **add learnable prefix vectors inside every transformer layer (attention layers)** to guide the model

Thank You